

W12D04 — Buffer: Testing Coverage

ML Engineer Journey | Phase 3 — Docker & Clean Architecture | 28 May 2026

1. Kenapa Testing Bukan Formalitas

Kamu sudah pasang alarm kebakaran (health/ready endpoint, graceful shutdown). Instalasi selesai, kabel terhubung, panel menyala. Tapi kamu tidak pernah tekan tombol test-nya. Gedung terbakar — alarm tidak bunyi. Ternyata kabel terpasang ke stopkontak yang mati.

Kode yang kelihatan benar tapi tidak pernah ditest adalah alarm itu. `/ready` yang harusnya return 503 saat Redis down — bagaimana kamu tahu itu benar-benar terjadi jika tidak ada test yang mensimulasikannya?

■ Test bukan bukti bahwa kode berjalan. Test adalah bukti bahwa kode berjalan dengan benar — termasuk pada kondisi yang tidak kamu harapkan.

2. Koneksi W12D03 → W12D04

Hari ini bukan topik baru dari nol — ini pembuktian bahwa semua yang dibangun di W12D03 benar-benar berfungsi sesuai kontrak:

W12D03 — Yang Dibangun	W12D04 — Yang Diverifikasi
app/core/state.py (AppState)	Nilai default benar, singleton satu instance
app/core/lifespan.py	ready=True terakhir saat startup, ready=False pertama saat shutdown
app/routers/health.py — /health	/health selalu 200 meski Redis down
app/routers/health.py — /ready	/ready 503 saat Redis ping gagal atau model None

3. Tiga Layer Testing di FastAPI

Testing di FastAPI dibagi tiga layer. Kita tidak bisa skip layer tengah (TestClient + mock) karena tanpanya tidak ada cara untuk mensimulasikan kondisi seperti Redis down atau app yang belum selesai startup.

Layer	Tools	Yang Ditest
Unit	pytest + assert langsung	AppState default values, singleton identity, flag mutation
Integration	TestClient + MagicMock	Endpoint behavior dengan state yang dikontrol (mock Redis, mock model)
Lifecycle	patch + call_order tracking	Urutan startup (ready=True terakhir), urutan shutdown (ready=False pertama)

4. conftest.py — Fondasi Semua Tests

AppState adalah singleton — satu objek memori untuk seluruh sesi pytest. Tanpa reset di antara test, state dari test A akan bocor ke test B (test pollution). `conftest.py` menyelesaikan ini dengan tiga fixture:

reset_app_state (autouse=True)

Dijalankan otomatis sebelum DAN sesudah setiap test tanpa perlu ditulis di parameter. Analogi: papan tulis yang selalu dihapus sebelum pelajaran baru dimulai. Tanpa ini, urutan test menentukan hasil — sangat berbahaya.

```
@pytest.fixture(autouse=True)

def reset_app_state():

    app_state.model_service = None # reset semua field

    app_state.redis_client = None

    app_state.ready = False

    app_state.startup_error = None

    yield # test berjalan di sini

    app_state.model_service = None # teardown: reset lagi

    app_state.redis_client = None

    app_state.ready = False
```

ready_client fixture

Fixture khusus yang mensimulasikan app yang sudah fully ready: `ready=True` dan Redis mock yang bisa `ping()`. Dipakai oleh test yang butuh skenario 'happy path'.

```
@pytest.fixture

def ready_client():

    app_state.ready = True

    mock_redis = MagicMock()

    mock_redis.ping.return_value = True # Redis sehat

    app_state.redis_client = mock_redis

    with TestClient(app) as c:
```

```
yield c
```

5. MagicMock — Dua Mode yang Berbeda Fungsi

MagicMock adalah pengganti object yang bisa dipanggil method apapun tanpa menyentuh resource sungguhan. Ada dua mode utama:

Mode	Syntax	Kapan Dipakai	Efek di Endpoint
Simulasi sukses	<code>mock.ping.return_value = True</code>	Happy path — Redis sehat	/ready lanjut, return 200
Simulasi gagal	<code>mock.ping.side_effect = RedisError(...)</code>	Redis timeout / putus	/ready catch exception, <code>checks[db]=False</code> , return
Simulasi urut	<code>mock.func.side_effect = [val1, val2]</code>	Return berbeda per panggilan	Jarang di sini, berguna untuk retry logic

6. Tiga Test yang Paling Kritis — dan Alasannya

A. /health selalu 200 meski Redis down

Ini adalah test kontrak utama /health. Kalau test ini fail, berarti seseorang menambahkan dependency check ke /health — yang tidak boleh ada di sana. Konsekuensi: container akan di-restart saat Redis down, padahal restart tidak membantu sama sekali karena Redis ada di server lain.

```
def test_health_always_200(self, client):

    app_state.ready = False

    app_state.redis_client = None # semua dependency mati

    app_state.model_service = None

    response = client.get('/health')

    assert response.status_code == 200 # HARUS tetap 200
```

B. /ready 503 saat Redis ping() gagal (bukan hanya None check)

Insight kritis dari W12D03: `redis_client is not None` ≠ Redis masih hidup. Object bisa ada (tidak None) tapi koneksi sudah putus. Analogi: punya nomor telepon seseorang tidak berarti mereka masih bisa dihubungi.

```
def test_ready_503_when_redis_ping_fails(self, client):

    app_state.ready = True # app 'siap' dari sisi state

    mock_redis = MagicMock()
```

```

mock_redis.ping.side_effect = redis.RedisError('timeout')

app_state.redis_client = mock_redis # object ada, tapi putus

response = client.get('/ready')

assert response.status_code == 503

assert response.json()['checks']['database_connected'] is False

```

C. /ready 503 tidak crash container (HTTPException vs bare Exception)

Ada perbedaan fundamental antara `raise HTTPException(503)` dan `raise Exception(...)`. `HTTPException` adalah respon yang dikelola FastAPI dengan clean — container tetap hidup. Bare Exception adalah crash — container restart. `TestClient` dengan `raise_server_exceptions=True` akan expose perbedaan ini.

```

def test_ready_503_does_not_restart_container(self, client):

    app_state.ready = False

    try:

        response = client.get('/ready')

        assert response.status_code == 503

    except Exception as e:

        pytest.fail(f'/ready raise exception yang tidak seharusnya: {e}')

```

7. Test Urutan Startup dan Shutdown

Ini adalah test yang paling sulit ditulis tapi paling bernilai. Kita tidak test return value — kita test urutan eksekusi. Caranya dengan melacak kapan setiap operasi terjadi ke dalam list, lalu verifikasi index-nya.

`app_state.ready = True` harus SELALU jadi baris terakhir startup. `app_state.ready = False` harus SELALU jadi baris pertama shutdown. Kenapa? Kalau `ready=True` dipasang terlalu awal → load balancer kirim traffic sebelum model dan Redis siap → user dapat error karena model belum ter-load. Kalau `ready=False` dipasang terlalu akhir → ada jeda waktu di mana app sedang close resource tapi request baru masih masuk.

```

call_order = []

def tracking_setattr(self, name, value):

```

```

if name == 'ready' and value is True:

    call_order.append('ready_set_true')

elif name == 'model_service' and value is not None:

    call_order.append('model_loaded')

...

# Verifikasi urutan

ready_index = call_order.index('ready_set_true')

assert call_order.index('model_loaded') < ready_index

```

8. Prinsip Kunci yang Harus Diingat

- **Test pollution adalah silent killer.**

AppState singleton menyimpan state sepanjang sesi pytest. Tanpa autouse=True di reset fixture, test yang lulus sendiri bisa gagal saat dijalankan bersama test lain karena mewarisi state yang salah.

- **Jangan test return value saja — test behavior.**

return_value = True hanya memverifikasi output. side_effect = RedisError memverifikasi bahwa sistem merespons dengan benar terhadap kegagalan. Production penuh dengan kegagalan — test harus mencerminkan itu.

- **Test 503 bukan test kegagalan — test bahwa sistem gagal dengan benar.**

503 yang benar menggunakan HTTPException. 503 yang salah menggunakan bare Exception dan menyebabkan container restart. Test ini memproteksi kamu dari kesalahan itu.

- **conftest.py adalah kontrak bersama seluruh test suite.**

Fixture di conftest.py berlaku untuk semua file test di direktori yang sama. Letakkan di sana semua yang dibutuhkan oleh lebih dari satu file test.

- **15/15 hijau bukan tujuan akhir — ini baseline.**

Setiap fitur baru yang kamu tambahkan di masa depan wajib datang dengan test baru. 15/15 hari ini bisa jadi 15/20 besok kalau ada code path baru yang tidak ditest.

9. Hasil Sesi — 15/15 Tests Hijau

Output pytest yang dicapai hari ini:

```

tests/test_health.py::TestHealthEndpoint::test_health_always_200 PASSED

tests/test_health.py::TestHealthEndpoint::test_health_response_structure PASSED

```

```
tests/test_health.py::TestHealthEndpoint::test_health_not_affected... PASSED

tests/test_health.py::TestHealthEndpoint::test_health_returns_app_version PASSED

tests/test_health.py::TestReadyEndpoint::test_ready_returns_503... PASSED

tests/test_health.py::TestReadyEndpoint::test_ready_returns_200... PASSED

tests/test_health.py::TestReadyEndpoint::test_ready_503_when_redis... PASSED

tests/test_health.py::TestReadyEndpoint::test_ready_response_includes... PASSED

tests/test_health.py::TestReadyEndpoint::test_ready_503_response_body... PASSED

tests/test_health.py::TestReadyEndpoint::test_ready_503_does_not... PASSED

tests/test_lifespan.py::test_ready_is_last_step_in_startup PASSED

tests/test_lifespan.py::test_ready_first_in_shutdown PASSED

tests/test_state.py::test_app_state_default_values PASSED

tests/test_state.py::test_app_state_singleton_identity PASSED

tests/test_state.py::test_ready_flag_mutation PASSED

15 passed in 0.22s
```

✓ Semua endpoint dari W12D03 terverifikasi. Kode tidak hanya 'kelihatan benar' — kini terbukti benar pada kondisi normal maupun kegagalan.